2026

Intelligent RAG-Driven Customer Support Ecosystem - Technical Documentation

DATA SCIENCE INTERNSHIP – FEBRUARY 2026

VEDIKA PUNJABI - IN226101802

Table of Contents

1. Introduction 2
   1.1 What is RAG 2
   1.2 Why it is needed 2
   1.3 Use Case Overview 2
2. System Architecture Explanation 2
   2.1 Detailed explanation of HLD 2
   2.2 Component Interactions 3
3. Design Decisions 3
   3.1 Chunk size choice 3
   3.2 Embedding strategy 3
   3.3 Retrieval approach 3
   3.4 Prompt design logic 3
4. Workflow Explanation 4
   4.1 LangGraph usage 4
   4.2 Node responsibilities 4
   4.3 State transitions 4
5. Conditional Logic 5
   5.1 Intent Detection 5
   5.2 Routing Decisions 5
6. HITL Implementation 5
   6.1 Role of Human Intervention 5
   6.2 Benefits and Limitations 5
7. Challenges & Trade-offs 6
   7.1 Retrieval Accuracy vs. Speed 6
   7.2 Cost vs. Performance 6
8. Testing Strategy 6
   8.1 Testing Approach 6
   8.2 Sample Queries 6
9. Future Enhancements 7
   9.1 Multi-Document Expansion 7
   9.2 Feedback Learning Loop 7
   9.3 Deployment 7

1. Introduction

1.1 What is RAG

- Retrieval-Augmented Generation (RAG) is a framework that optimizes LLM output by referencing a specific, authoritative knowledge base.
- It allows the model to act as a reasoning engine over external data rather than relying solely on its static training set.

1.2 Why it is needed

- Standard LLMs often "hallucinate" or provide outdated information when asked about niche or private organizational data.
- RAG is essential for accuracy in professional environments, ensuring every answer is grounded in verified, real-world facts.

1.3 Use Case Overview

- This project serves as a smart support assistant for the University of Arkansas Extension, handling complex policy queries.
- The system automates routine answers while escalating high-stakes or missing information to a human expert.

2. System Architecture Explanation

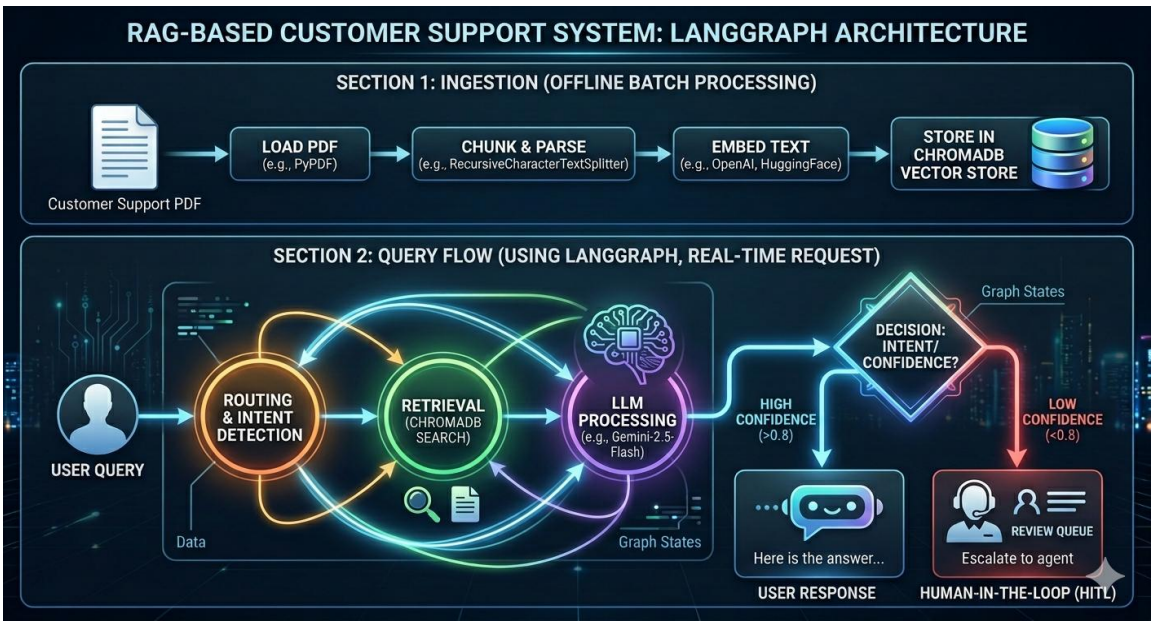

Figure 1: Dual-Phase RAG Architecture & LangGraph Workflow Logic

2.1 Detailed explanation of HLD

- The High-Level Design follows a dual-stage architecture: an Ingestion Pipeline for batch processing and a Real-Time Query Flow for interaction.

- This modular approach separates data preparation from data retrieval, allowing for independent scaling of the vector store and the reasoning engine.

2.2 Component Interactions

- **LangGraph** acts as the central orchestrator, managing the "State" object that carries conversation history across nodes.
- The system facilitates a seamless handshake between the **ChromaDB** vector store and **Gemini 2.5 Flash Lite**, ensuring the LLM always has relevant context before responding.

3. Design Decisions

3.1 Chunk size choice

- We utilized a chunk size of 1000 characters with a 150-character overlap using the `RecursiveCharacterTextSplitter`.
- This size ensures technical policy steps are not cut in half, while the overlap prevents critical context from being lost at the boundaries of a split.

3.2 Embedding strategy

- The system employs `gemini-embedding-001`, converting text into 768-dimensional dense vectors.
- This model ensures semantic alignment with Gemini 2.5 Flash Lite, providing superior retrieval accuracy within the Google AI ecosystem.

3.3 Retrieval approach

- A Similarity Search ($k = 3$) was implemented to fetch the three most relevant text segments for every user query.
- This configuration balances context richness with token efficiency, preventing "noise" from irrelevant document sections.

3.4 Prompt design logic

- A "Strict Grounding" system prompt mandates that the AI only answer if the solution is explicitly found in the provided context.
- The prompt is engineered to output a specific `ESCALATE` token if the information is missing, serving as the trigger for the human review path.

4. Workflow Explanation

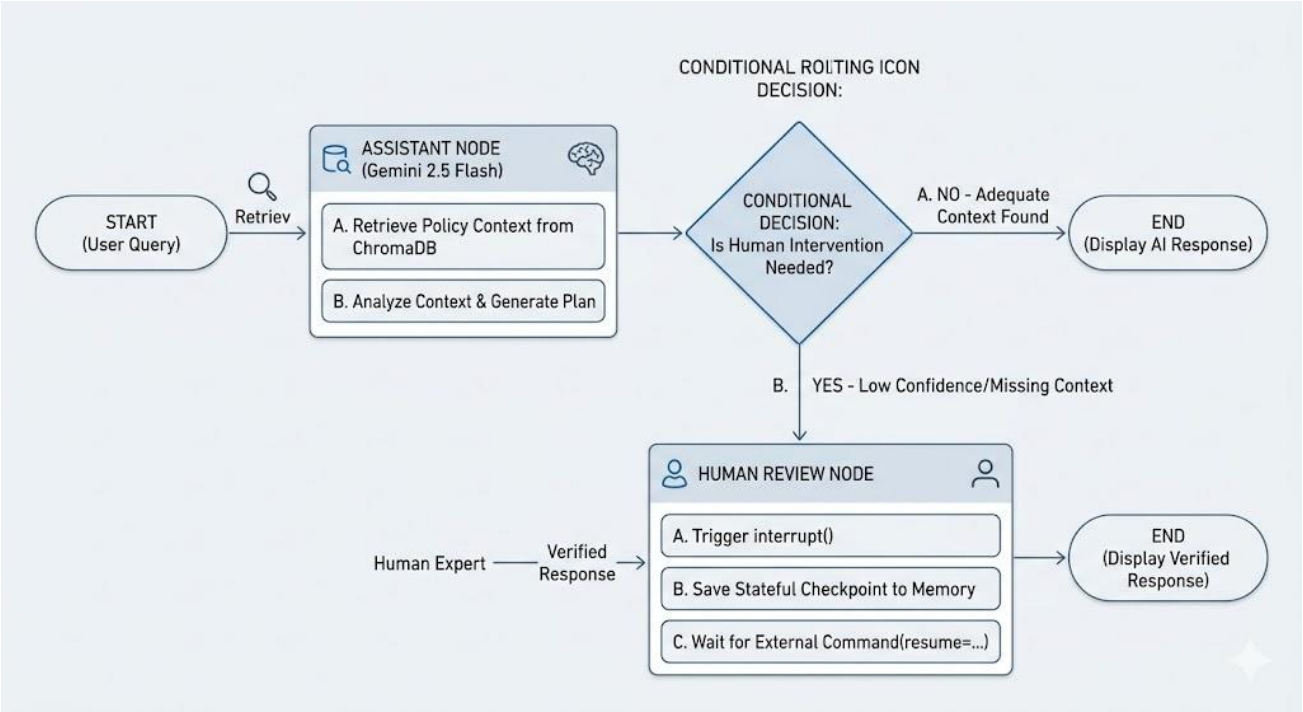

Figure 2: LangGraph Stateful Execution Flow showing automated retrieval and human-in-the-loop transition.

4.1 LangGraph usage

- LangGraph enables stateful, cyclic workflows that standard linear chains cannot support.
- It provides the mechanical basis for the "interrupt" feature, allowing the system to pause and wait for external human input without losing state.

4.2 Node responsibilities

- The **Assistant Node** manages the retrieval of chunks and the generative reasoning of the LLM.
- The **Human Review Node** functions as a gatekeeper that triggers when the AI hits its confidence threshold, requiring manual verification.

4.3 State transitions

- The workflow starts at **START** → **Assistant**; based on logic, it then routes to either **END** or **Human Review**.
- State is persisted via a `MemorySaver` checkpointer, allowing the human to resume the exact thread where the AI left off.

5. Conditional Logic

5.1 Intent Detection

- The system performs implicit intent analysis to see if the user's question aligns with the Arkansas Extension policy handbook.
- If the query intent (e.g., "cafeteria menu") falls outside the document scope, the system identifies this gap instantly.

5.2 Routing Decisions

- A Python-based router evaluates the model's internal `needs_escalation` flag.
- If the flag is `True`, the graph reroutes to the human queue; if `False`, it delivers the direct AI answer.

6. HITL Implementation

6.1 Role of Human Intervention

- The human expert acts as the final authority, providing "Ground Truth" answers for queries the AI cannot confidently solve.
- This ensures that the user never receives an incorrect answer, even if the primary documentation is incomplete.

6.2 Benefits and Limitations

- The primary benefit is 100% factual reliability and risk mitigation for the organization.
- The limitation is added latency, as the response time is dependent on human availability for escalated queries.

7. Challenges & Trade-offs

7.1 Retrieval Accuracy vs. Speed

- We prioritized **ChromaDB** for local, sub-second retrieval speeds over slower, cloud-based semantic rankers.
- This ensures a snappy user experience for the majority of queries that are found directly in the text.

7.2 Cost vs. Performance

- Gemini 2.5 Flash Lite was chosen for its low latency and cost-effectiveness compared to the larger "Pro" models.
- This choice provides professional-grade reasoning while keeping the operational overhead low for a support tier.

8. Testing Strategy

```python
from langgraph.types import Command

# 3. Check if the graph is waiting for a human
snapshot = app.get_state(config)

if snapshot.next:
    print(f"\n[!] SYSTEM PAUSED at node: {snapshot.next}")
    print("The AI has requested human intervention.")

    # 4. Provide the manual expert response
    # You can look up the answer in the Arkansas PDF (e.g., 'Return calls within 24 hours')
    human_expert_answer = input("Enter the expert response to provide to the user: ")

    # 5. RESUME the graph using Command(resume=...)
    print("\n--- RESUMING WITH EXPERT INPUT ---")
    for event in app.stream(Command(resume=human_expert_answer), config, stream_mode="values"):
        if "messages" in event:
            print(f"Final Verified Response: {event['messages'][-1].content}")
else:
    print("\n[√] The Assistant answered successfully using the PDF context.")
```

```
[!] SYSTEM PAUSED at node: ('human_review',)
The AI has requested human intervention.
Enter the expert response to provide to the user:
```

8.1 Testing Approach

• "Adversarial Testing" was used to verify that the AI doesn't guess when it doesn't know.

• We asked questions both inside the scope (Phone Etiquette) and outside the scope (Office Menus) to validate the routing logic.

8.2 Sample Queries

- *In-Scope:* "How should I handle a disgruntled caller?" → (Success: Correct policy answer).
- *Out-of-Scope:* "What are the rules for the company parking lot?" → (Success: Correctly escalated to Human).

9. Future Enhancements

9.1 Multi-Document Expansion

- Future versions will index the entire University library to handle cross-departmental questions.

9.2 Feedback Learning Loop

- We plan to store human corrections back into the vector database, allowing the AI to "learn" and handle those queries autonomously next time.

9.3 Deployment

- Containerizing the solution with Docker for deployment on cloud platforms like GCP or AWS.